

Advanced Database Labs

ADB Labs

Amira Nasreldeen, Aseel, Salma

© 2025 Amira – Aseel – Salma

Table of contents

1. Advanced Database Labs	3
1.1 What You Will Learn	3
1.2 Tutorial Support video	3
1.3 Labs Overview	3
1.4 What You Should Do	3
2. General Instructions	4
2.1 Please follow these instructions carefully:	4
3. Setup environment	5
3.1 Prerequisites: Install Microsoft Visual C++ 2019 Redistributable	6
3.2 MySQL	6
3.3 Python programming language	12
3.4 Alternative way to practice:	14
3.5 Assignment: Install MySQL, Create a Database, and Query It	14
4. Transaction Management and SQL Implementation	15
4.1 Understanding SQL Transactions	15
4.2 Assignment: Simulate The Unrepeatable Read Problem	25

1. Advanced Database Labs

Welcome to the Advanced Database Lab course. This site provides hands-on labs to help you learn and simulate key concepts in **concurrency control** and **database recovery techniques**.

1.1 What You Will Learn

- How to install and use **MySQL**, **Python**, and **Visual Studio Code**
- An understanding of **transaction management** and **SQL implementation**
- Understanding concurrency control mechanisms (**2PL**, **Timestamp ordering**)
- Understanding Database recovery techniques (**ARIES**)
- An introduction to **database security** practices:
- **Role-Based Access Control (RBAC)**
- **Data encryption**

1.2 Tutorial Support video

Many labs include **video walkthroughs** that demonstrate how to solve key theoretical examples step by step.

- Transaction schedules and conflicts
- Concurrency Control mechanisms
- Database recovery Techniques

Be sure to watch them alongside the written material for deeper understanding.

1.3 Labs Overview

Each lab builds on the previous one. Please complete them in sequence:

1. **Lab 0:** Environment Setup
2. **Lab 1:** Basics of Transaction Management and Its Properties
3. **Lab 2:** Concurrency Control Mechanisms
4. **Lab 3:** Database Recovery Techniques (ARIES)
5. **Lab 4:** Database Security Measures



Note

You can download a single PDF that contains all labs. It will be updated each time a new lab is uploaded.

[Download"](#)

1.4 What You Should Do



Each lab includes an assignment that you are expected to complete as part of the learning process. **Please submit the assignments on time.**

2. General Instructions

2.1 Please follow these instructions carefully:

2.1.1 File Submission

- At the top of your document, please include the following information:
- Full Name
- Department
- Index Number
- Submit your assignment as a single PDF file.
- The PDF must include all required screenshots (code and output).

2.1.2 Lab Title

Write the lab title as "Lab 0, Lab 1, ..." clearly at the top of your PDF.

2.1.3 Screenshot Annotations

On every screenshot containing SQL script, add a comment at the top with the following information:

1. Student Name
2. Department
3. Student Index Number

```
-- Name: student name  
-- Department: Computer Science, ...  
-- Index: 01x-xxx
```

2.1.4 Deadline

1. Submissions must be uploaded before the deadline.
2. Late submissions may be penalised according to the situations.

2.1.5 Plagiarism

- You may discuss the assignment with other students, but all work must be completed individually.
- Plagiarism will be identified and penalized appropriately based on the circumstances.
- The use of AI tools to complete your tasks is not allowed. If detected, it will result in penalties.

3. Setup environment

Objective:

- To set up **MySQL Database Management System**
- To connect to **MySQL** server
- To set up **Python Programming Language**

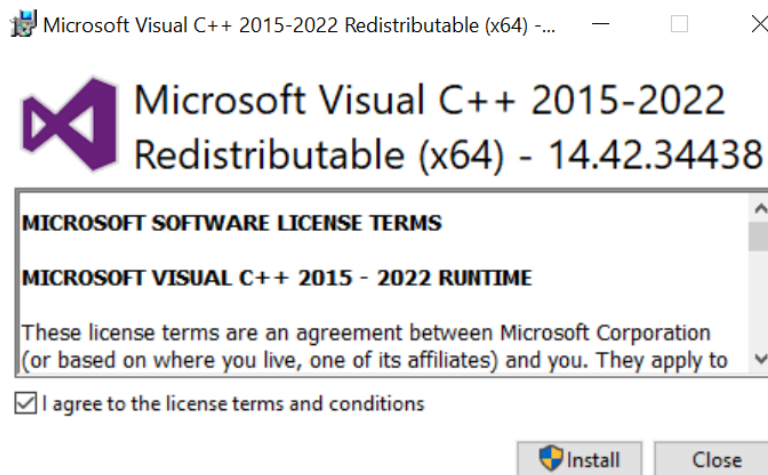
Software:

- MySQL installer (offline version)
- Visual Studio code (VS code)

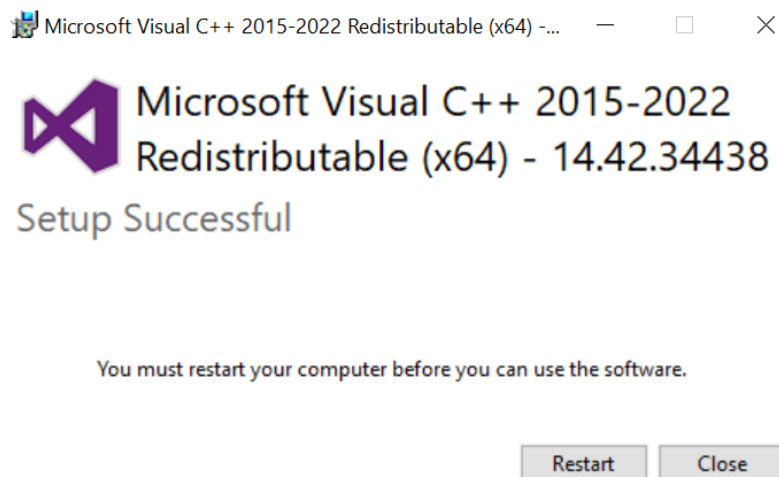
3.1 Prerequisites: Install Microsoft Visual C++ 2019 Redistributable

If you already have Microsoft Visual C++ 2019 Redistributable installed, you can skip this step and continue with the lab.

1. Go to the official Microsoft page. [Download VC++ Redistributables](#)
2. Download the file based on your system:
 - for 64-bit [VC_redist.x64.exe](#)
 - for 32-bit [VC_redist.x86.exe](#)
3. Open the downloaded file and check on **I agree to the license terms and conditions**.



4. After installation, **restart your PC**.



3.2 MySQL

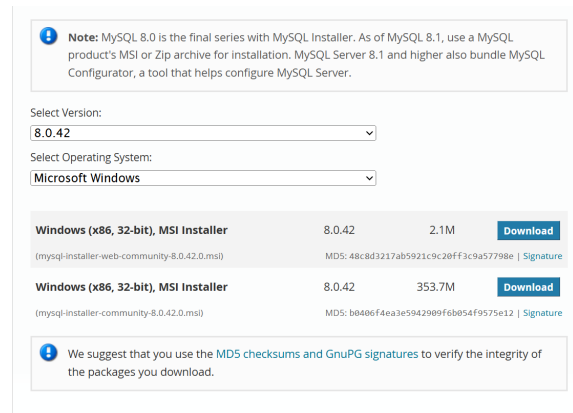
MySQL is an open-source relational database management system (RDBMS) that uses SQL to create, manage, and query data.

3.2.1 Why we will use MySQL?

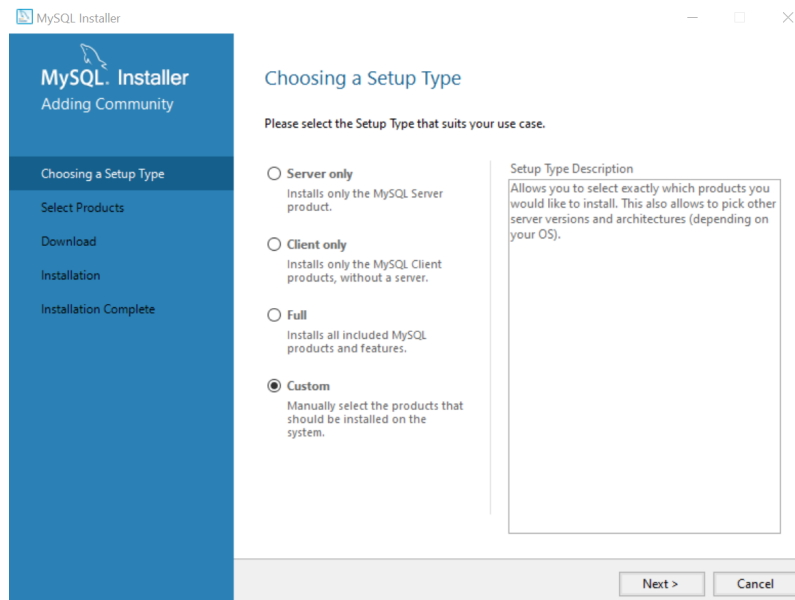
1. Support **ACID transaction properites**, essential for understanding transaction management
2. Uses the **InnoDB** transactional storage engine, which supports **concurrency control** and **recovery mechanisms**
3. Flexible and easy-to-use

3.2.2 Installing MySQL (Offline)

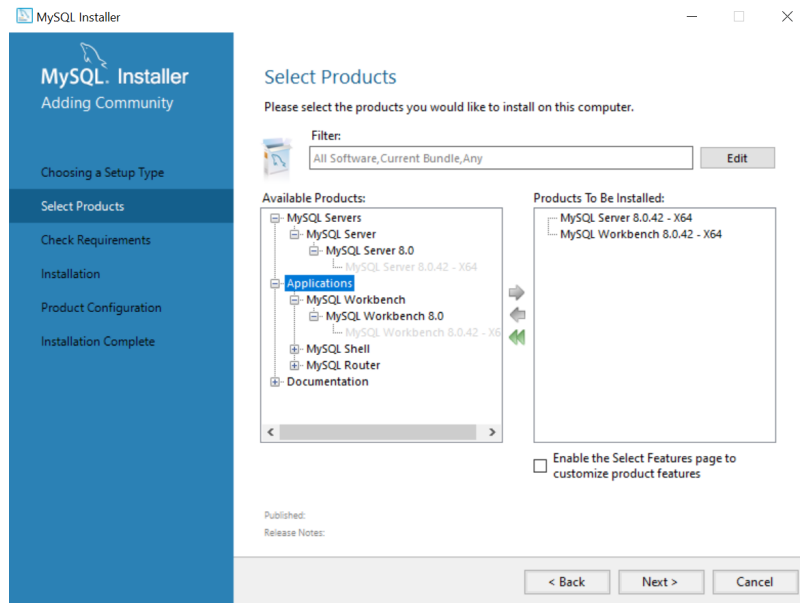
1. Go to the official [MySQL Downloads Page](#)
2. Download the **Windows (x86, 32-bit), MSI Installer** (Offline version ~353.7M)



3. After open the Installer Choose **Custom**

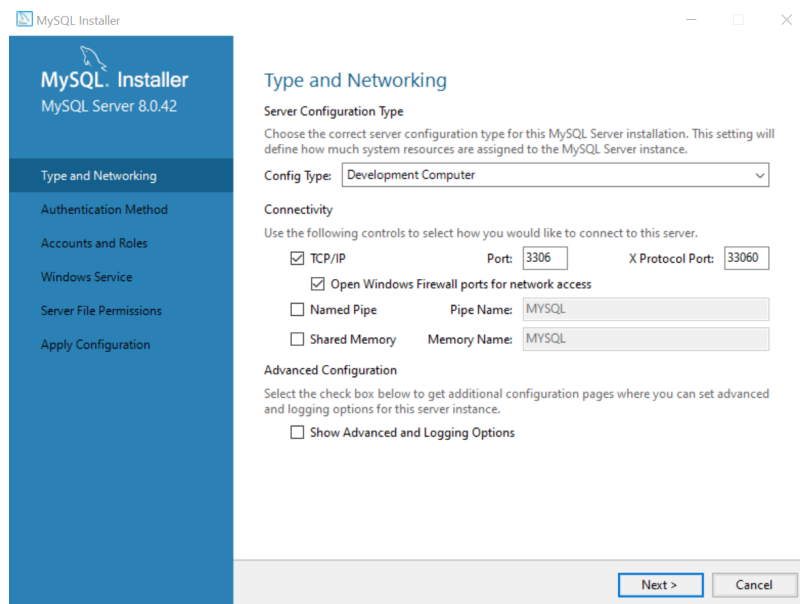


4. Select **MySQL Servers** and **MySQL Workbench**

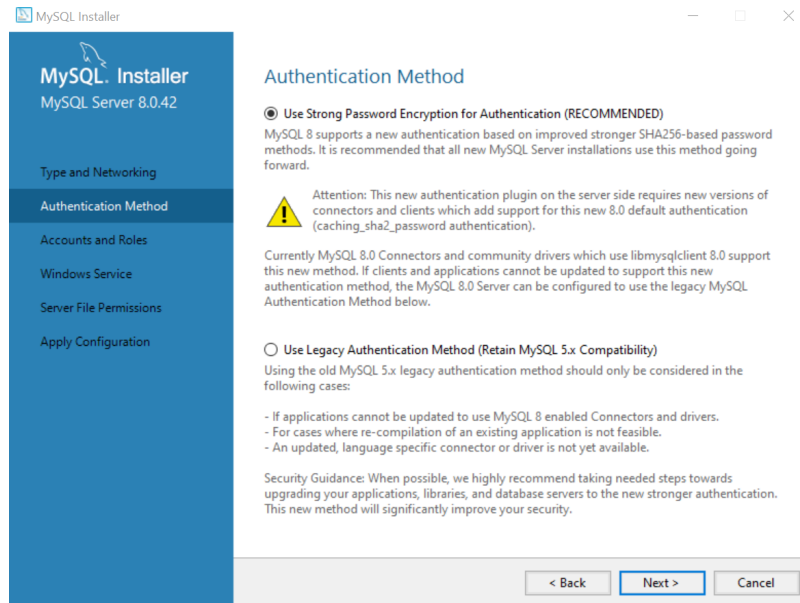


5. Proceed through the installation wizard

6. Use the default port (3306)

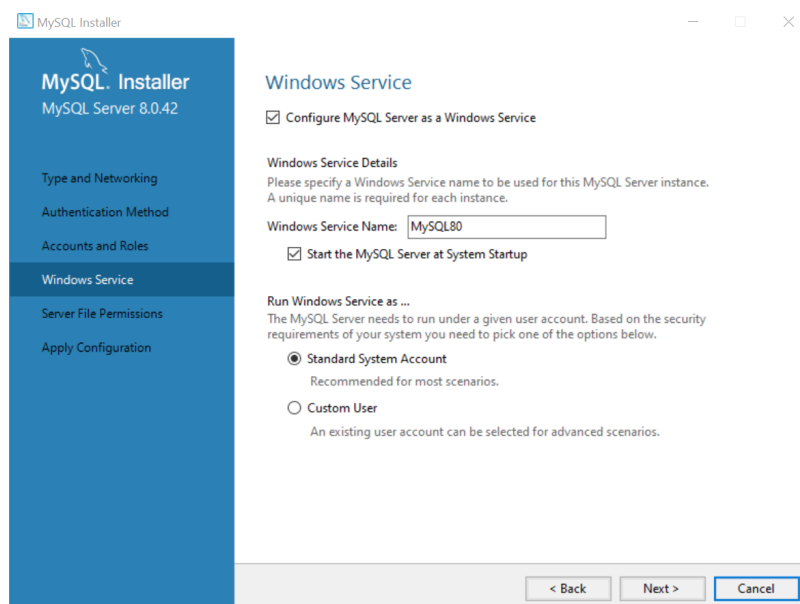


7. Choose the Authentication Method as below



8. Create a root password

9. Choose Windows Service as below



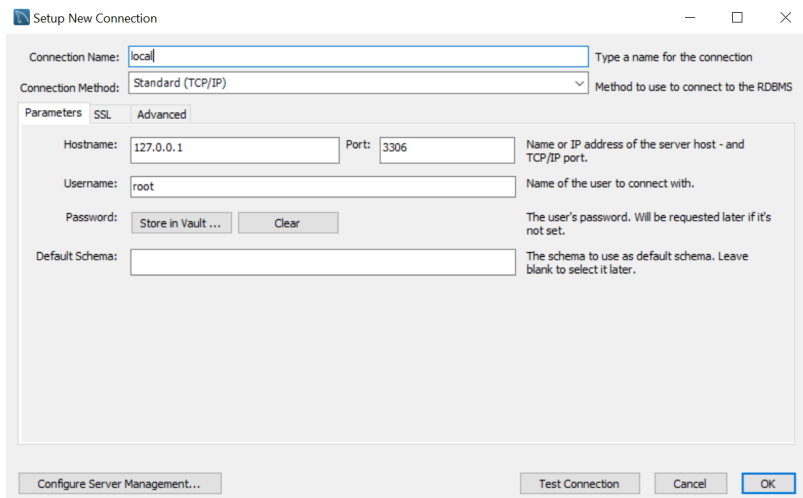
10. Proceed through the installation wizard until finish

3.2.3 What after install MySQL?

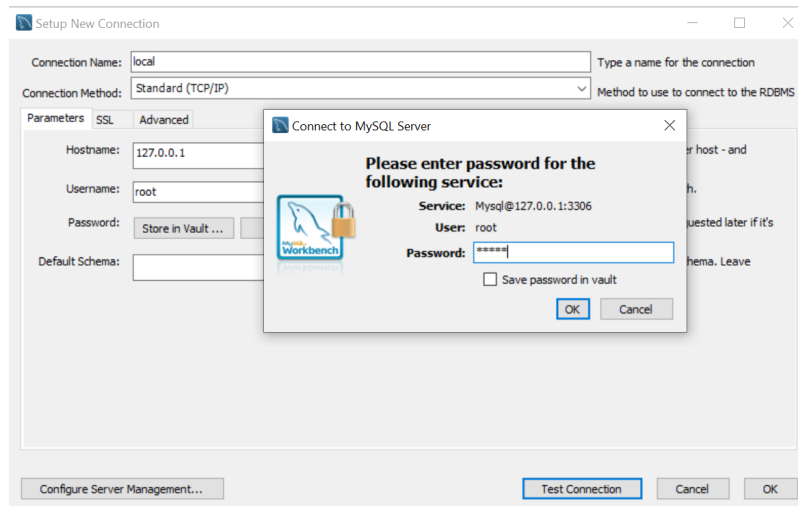
1. After installing MySQL, connect to the **MySQL Server**. by clicking the **+** symbol next to MySQL Connections



2. In the dialog box, enter a **Connection name**, e.g., `Local`

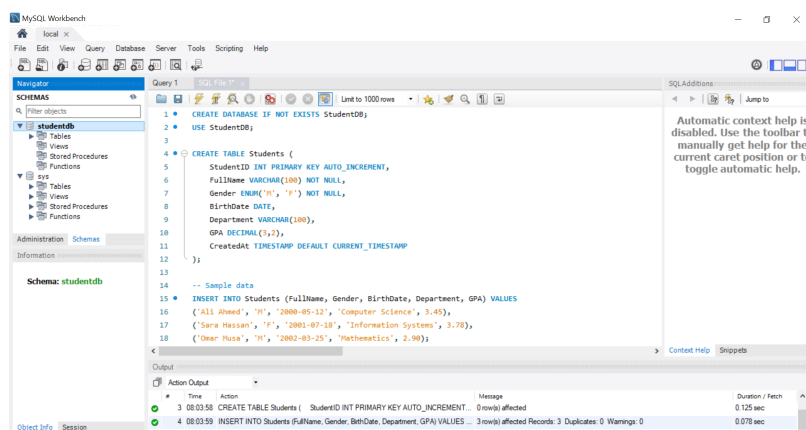


3. Click **Test Connection** and enter your MySQL root password to verify the connection



4. After successfully connecting to the MySQL server, create a new database called `StudentDB` :

- Open `local` connection
- Click **Create new SQL** from the top-left corner
- Write SQL commands to create the `StudentDB` database, a `Students` table, and insert sample data.
- Execute the Script using Execute button 



```

/*
Name: Amira Naser Aldeein
Index: 01x-xxx
Department: e.g., CS
*/

CREATE DATABASE IF NOT EXISTS StudentDB;
USE StudentDB;

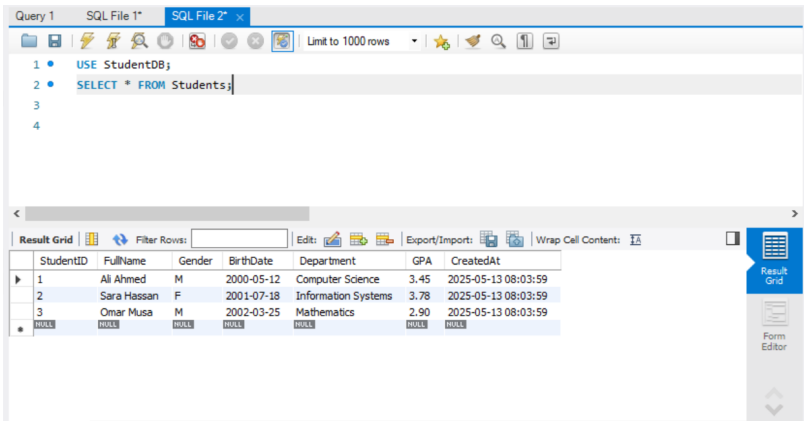
CREATE TABLE Students (
  StudentID INT PRIMARY KEY AUTO_INCREMENT,
  FullName VARCHAR(100) NOT NULL,
  Gender ENUM('M', 'F') NOT NULL,
  BirthDate DATE,
  Department VARCHAR(100),
  GPA DECIMAL(3,2),
  CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Sample data
INSERT INTO Students (FullName, Gender, BirthDate, Department, GPA) VALUES
('Ali Ahmed', 'M', '2000-05-12', 'Computer Science', 3.45),
('Sara Hassan', 'F', '2001-07-18', 'Information Systems', 3.78),
('Omar Musa', 'M', '2002-03-25', 'Mathematics', 2.90);

```

 Once you create the database, you will see it listed under the Schemas section on the left panel

5. Now you can run SQL queries on your new database



```
USE StudentDB;
SELECT * FROM Students;
```

What do we mean by Connection and Port?

A database connection is a communication link between a client application (e.g., MySQL Workbench) and the MySQL server. When an application (the client) needs to perform operations such as creating databases, running queries, or retrieving data, it establishes a connection to communicate with the server. The port is a communication endpoint on your machine. By default, MySQL uses port 3306 to listen for incoming client requests. In short, a connection is essential to allow your client tools or applications to interact with the MySQL server, run queries, manage data, and exchange information.

3.3 Python programming language

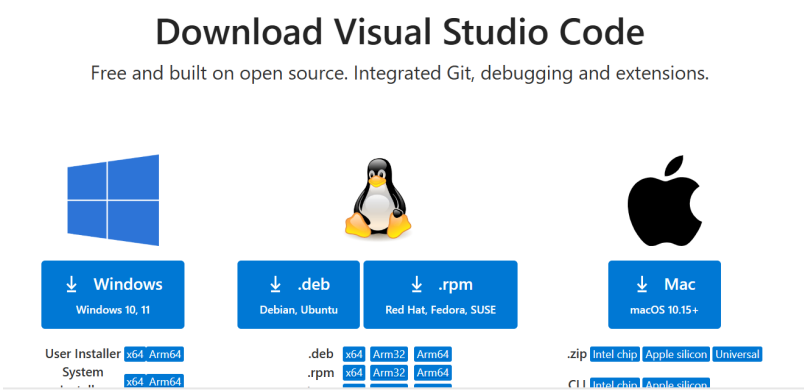
We will use the **Python programming language** to simulate the following: * Concurrency control algorithms * Database recovery techniques

3.3.1 Download and Install Python

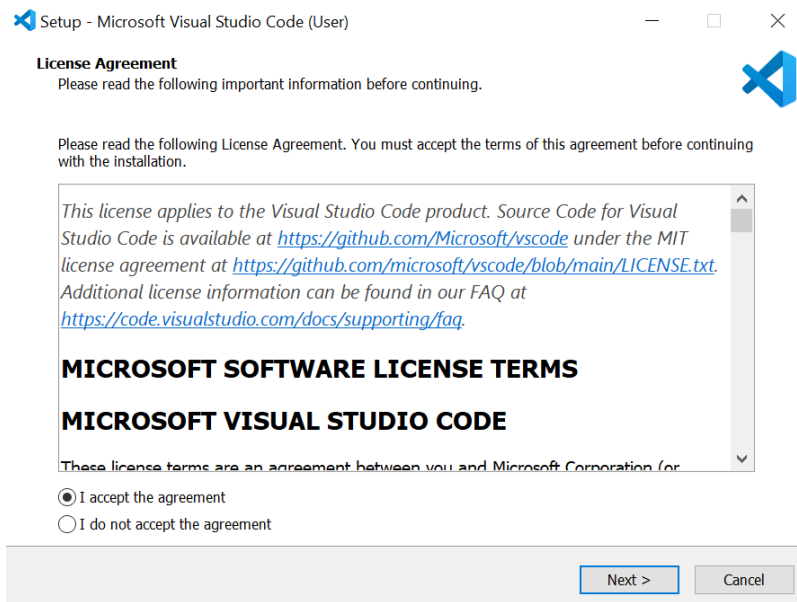
- 1. Go to the **Python** official website
- 2. Download the latest stable version
- 3. Open the installer and check
 - Add Python to PATH

3.3.2 Download and Install VS Code

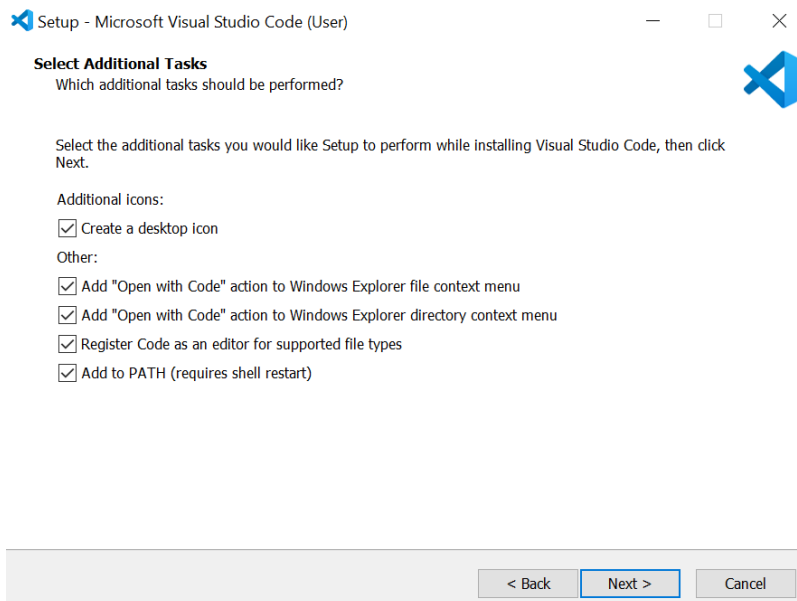
- 1. Visit the website **VS code** Click on **Windows** to download VS code for Windows, as shown below



- 2. After the download finished open the `VSCodeUserSetup` executable file, when it open select `I accept the agreement` then click on `Next`



3. Select all the option as seen below



4. Finally we are ready to install the VS code, click on `install` and wait until the setup finish

5. On the Vs code, on the left-hand side, click on `extension` then in the search bar, write `Python` select `Python microsoft` and click on `Install` as shown below



3.4 Alternative way to practice:

* Online Postgres server **[supabase](https://supabase.com/)**
 * Online Python Editor **[programiz](https://www.programiz.com/python-programming/online-compiler/)**

3.5 Assignment: Install MySQL, Create a Database, and Query It

 Due Date on 24/5/2025

See the requirement about the structures of the lab [here](#)

1. Download and Install MySQL
2. Connect to the server
3. Create a New Database
4. Create a Table
5. Insert Data
6. Query the Data

```
USE DatabaseName;
SELECT * FROM TableName;
```

3.5.1 What to assign:

Take Screenshots:

1. SQL code (on MySQL)
2. The output of your query
3. Put it all on one document

4. Transaction Management and SQL Implementation

Objective:

- Understand the ACID properties.
- Implement SQL transactions.
- Understand transaction control.

4.1 Understanding SQL Transactions

A **Transaction** is a sequence of read and write operations performed as a single logical unit of work.

SQL Server operates in the following transaction modes:

1. Autocommit transactions: Each individual statement is a transaction.
2. Explicit transactions: Each transaction is explicitly started with the `BEGIN TRANSACTION` statement and explicitly ended with a `COMMIT` or `ROLLBACK` statement (**which is concerned in this course**).
3. Implicit transactions: A new transaction is implicitly started when the prior transaction completes, but each transaction is explicitly completed with a `COMMIT` or `ROLLBACK` statement.

4.1.1 Transaction Control Commands – `SAVEPOINT`, `ROLLBACK`, and `COMMIT`

Objective: Understand how to manage multi-step database transactions using SQL transaction control commands.

BEGIN

Start a new transaction:

```
BEGIN;  
-- or  
START TRANSACTION;
```

SAVEPOINT

Set a point you can roll back to later:

```
BEGIN;  
  
INSERT INTO employees (id, name, department) VALUES (1, 'Alice', 'HR');  
SAVEPOINT sp1;  
  
INSERT INTO employees (id, name, department) VALUES (2, 'Bob', 'Finance');  
SAVEPOINT sp2;  
  
INSERT INTO employees (id, name, department) VALUES (3, 'Charlie', 'IT');
```

At this point, we have added 3 employees and created two savepoints: `sp1` and `sp2`.

ROLLBACK

Undo part or all of a transaction.

ROLLBACK TO A SAVEPOINT

```
ROLLBACK TO sp2;
```

This undoes only the insertion of **Charlie**, keeping **Alice** and **Bob**.

ROLLBACK ENTIRE TRANSACTION

```
ROLLBACK;
```

This undoes **all** changes made in the transaction.

COMMIT

Make all changes permanent:

```
COMMIT;
```

After committing, the changes cannot be undone.

Summary

- Use `SAVEPOINT` to mark logical checkpoints.
- Use `ROLLBACK` to undo mistakes.
- Use `COMMIT` to make sure the database saves your work.

This gives you control and safety when working with important or complex data operations.

4.1.2 Transaction properties and how SQL supports them

Transaction properties (ACID):

1. **A - Atomicity:** All actions in transaction happen, or none happen. “*All or nothing...*”
2. **C - Consistency:** If each transaction is consistent and the database starts in a consistent state, it ends in a consistent state. “*It looks correct to me...*”
3. **I - Isolation:** Execution of one transaction is isolated from other transactions. “*All by myself...*”
4. **D - Durability:** Once a transaction commits, its effects are permanent. “*I will survive...*”

How SQL Supports Each Property?

properties	SQL Feature
Atomicity	Use <code>START TRANSACTION</code> , followed by <code>COMMIT</code> or <code>ROLLBACK</code>
Consistency	Enforced through constraints (e.g., <code>PRIMARY KEY</code> , <code>FOREIGN KEY</code> , <code>CHECK</code>)
Isolation	Set using <code>SET TRANSACTION ISOLATION LEVEL</code>
Durability	Ensured by <code>COMMIT</code> and Logging mechanisms (e.g., write-ahead logs, redo logs)

4.1.3 Why Concurrency Control is Needed

Several problems can occur when concurrent transactions execute without proper control, especially due to conflicting operations such as **Read-Write**, **Write-Read**, and **Write-Write** conflicts.

Here is an illustrated problem with SQL transactions to simulate the problem:

1. Lost update problem

This problem occurs when two or more transactions read and update the same data item concurrently without being aware of each other's changes. As a result, the last update overwrites the previous ones, leading to the loss of some updates and causing data inconsistency.

Real-World Scenario

Two users (sessions) try to withdraw money from the same account at the same time:

- **User A** reads the balance = 100, decides to withdraw 30 → wants to update it to 70.
- **User B** also reads the balance = 100, decides to withdraw 50 → wants to update it to 50.
- If both updates happen without proper control, one will overwrite the other, leads to lost updates and incorrect final balance.

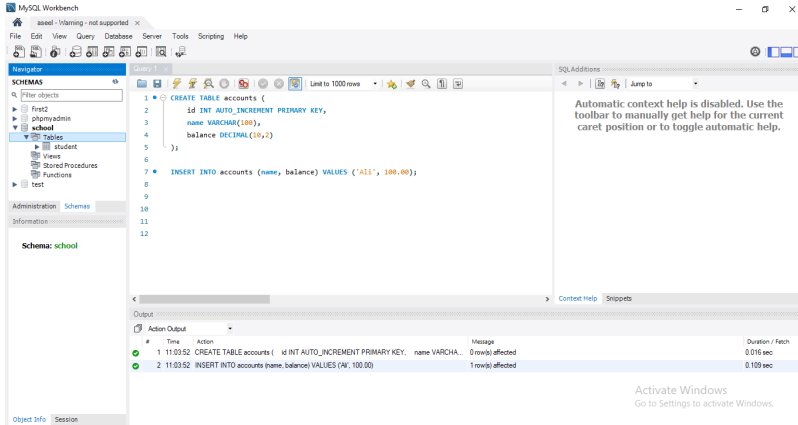
Example of the Scenario in SQL:

To simulate a Lost Update in MySQL using two concurrent sessions:

1. Create a Table: Create any Database and create a table named `accounts` and insert sample data into it:

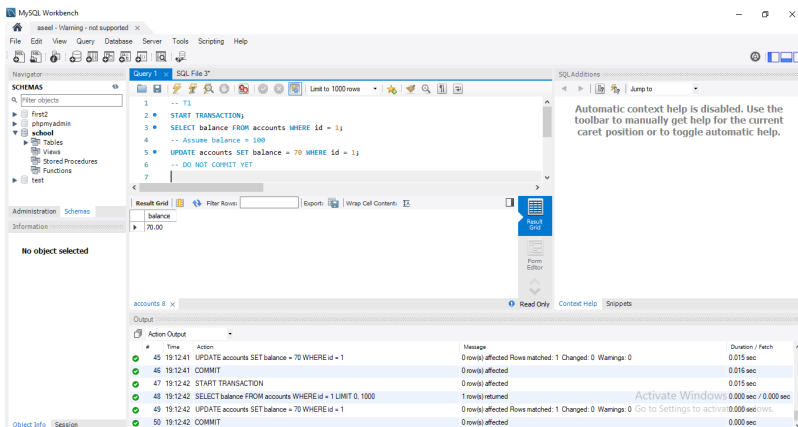
```
CREATE TABLE accounts (
  id SERIAL PRIMARY KEY,
  name TEXT,
  balance NUMERIC(10,2)
);

INSERT INTO accounts (name, balance) VALUES ('Ali', 100.00);
```



2. Session A (user A):

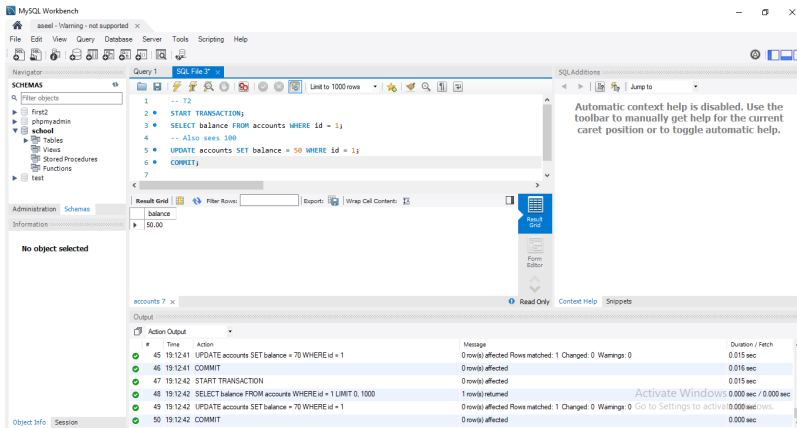
```
START TRANSACTION;
SELECT balance FROM accounts WHERE id = 1;
UPDATE accounts SET balance = 70 WHERE id = 1;
```



Here, user A reads the balance (100) and calculates the new balance: $100 - 30 = 70$, then updates the record without commit.

3. Session B (user B, in another window):

```
START TRANSACTION;
SELECT balance FROM accounts WHERE id = 1;
UPDATE accounts SET balance = 50 WHERE id = 1;
COMMIT;
```

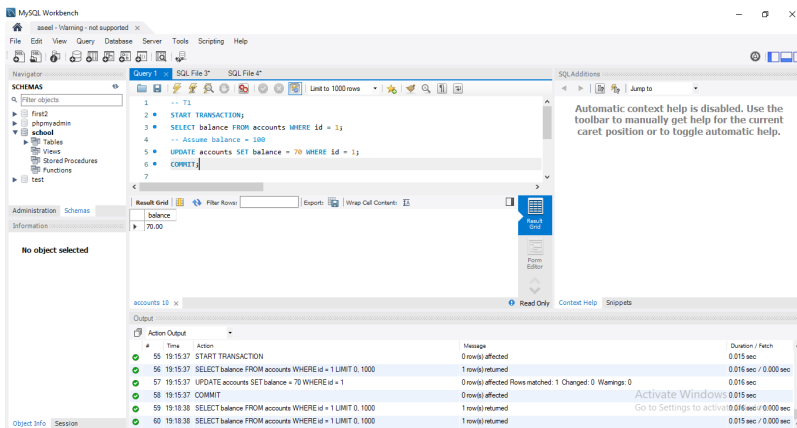


Here, user B reads the balance (100) and calculates the new balance:

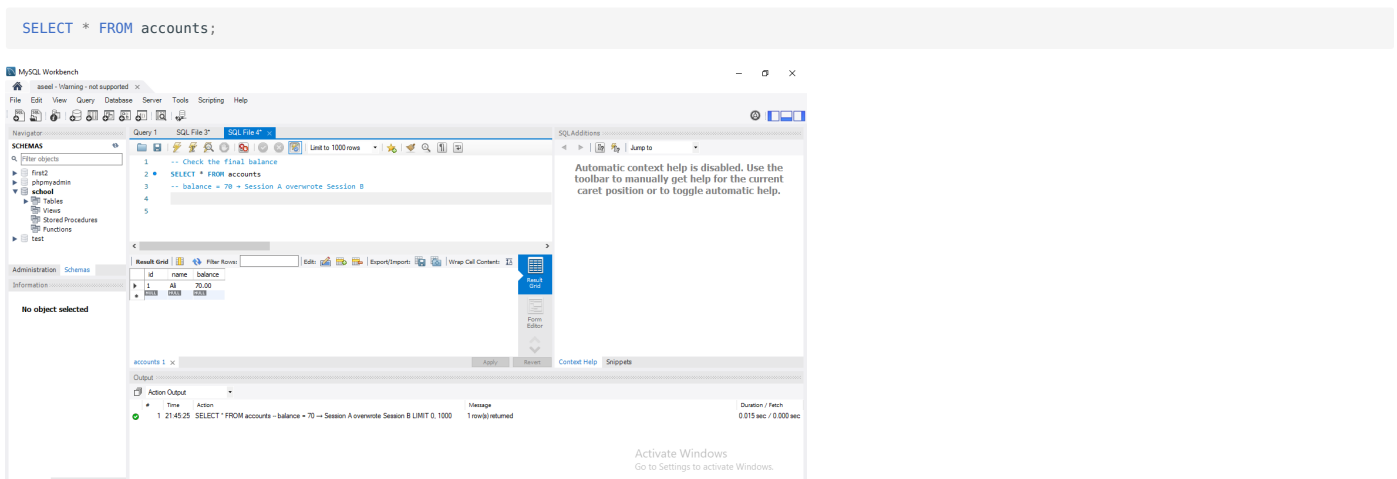
$100 - 50 = 50$, then updates the record and commit.

to open two sessions in MySQL Workbench; Go to File → New Query Tab to open Session A, Repeat to open another tab (Session B). Each tab is a separate session — you can run SQL commands independently in each

4. Back to session A: Now return back to session A again and commit the session:



5. Check the final balance:



Here we will see the balance becomes 70, and user B's update (which should make the balance 50) is lost.

6. Conclusion:

- Both transactions read the same initial value (100).
 - Each transaction updates the data without being aware of the other's changes.
 - The last transaction to commit determines the final value, possibly erasing the other's update.
 - Even though no errors appear, data integrity is lost. This demonstrates the importance of proper concurrency control in database systems.
-

2. The Temporary Update (or Dirty Read) Problem

This occurs when one transaction reads data that has been written by another transaction that hasn't committed yet. If the first transaction is later **rolled back**, the second transaction ends up working with invalid or temporary data — a dirty read.

Example Scenario (Simulated):

In an online electronics store:

1. Ahmed initiates a purchase of 200 laptops. A transaction begins and temporarily reduces the stock quantity to 0, but the transaction is not yet committed.
2. Meanwhile, Mohamed checks the available stock and sees 0 laptops — based on User A's uncommitted change.
3. Then, Ahmed cancels the purchase, and the transaction is rolled back. The stock returns to its original quantity (e.g., 200).
4. However, Mohamed made a decision based on incorrect (temporary) data — the system gave them a dirty read.

SQL Transaction for the scienario:

1. Create the `Products` table to simulate the scenario

```
-- Create ProductDB Database
CREATE DATABASE IF NOT EXISTS ProductDB;
USE ProductDB;

-- Create Products table
CREATE TABLE Products (
  Id INT PRIMARY KEY,
  ProductName VARCHAR(100),
  Quantity INT
);

-- Insert Sample data
INSERT INTO Products (Id, ProductName, Quantity) VALUES
(1, 'Mobile', 100),
(2, 'Tablet', 50),
(3, 'Laptop', 200);
```

2. In Session 1: Connect using the previous connection from **Lab0** (named local), and open a new SQL query tab

```
USE ProductDB;

-- Transaction 1: Simulate purchase
START TRANSACTION;
-- Pay 200 units from product with ID = 3 (assumed to be a laptop)
UPDATE Products
SET Quantity = Quantity - 200
WHERE Id = 3;
-- Simulate waiting for payment confirmation
DO SLEEP(15);
-- Payment failed – rollback the transaction
ROLLBACK;
```

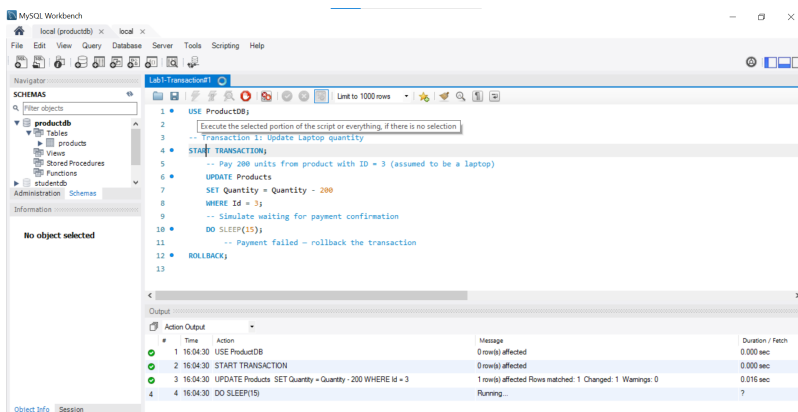
We use `SLEEP` to simulate concurrent transactions. Note that `DO SLEEP(15)` works only in MySQL 8.0.29 or later

3. Go to the Home tab and open the `local` connection again. (This step simulates a second session connected to the database.) Then, open a new SQL query tab to continue.

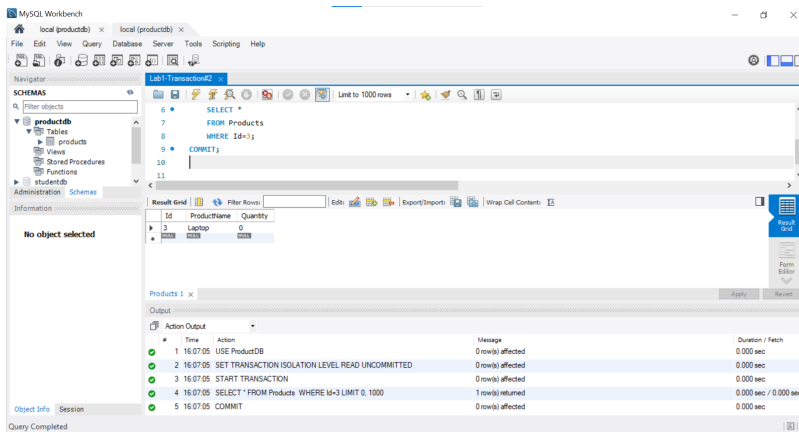
```
USE ProductDB;

-- Transaction 2: Retrieve Laptop record
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
START TRANSACTION;
SELECT *
FROM Products
WHERE Id=3;
COMMIT;
```

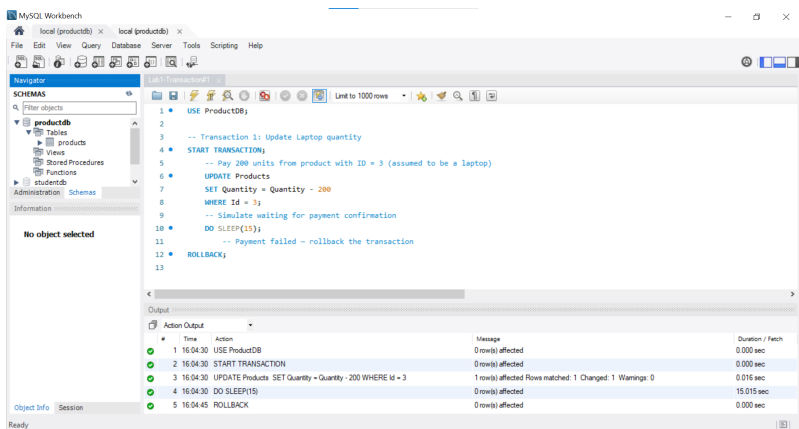
4. In Session 1: Run the transaction and wait



5. While Session 1 is still sleeping: Run Session 2



6. After Session 1 completes and rolls back



7. Conclusion: Session 2 was able to see the updated quantity (0 laptops) before Session 1 rolled back. This leads to inconsistent or misleading data, which is why isolation levels like `READ COMMITTED` or `REPEATABLE READ` are important in real-world applications.

3. The Incorrect Summary Problem

Problem Statement

When you use aggregate functions like `SUM`, `AVG`, and `COUNT` on a table while other transactions are modifying the data, the result may be **inconsistent**. This is because some rows might be updated **before** the function reads them, and others **after**.

1. Create the Table

```
CREATE TABLE accounts (
  id INT PRIMARY KEY,
  balance DECIMAL(10,2)
);
```

2. Insert Initial Data

```
INSERT INTO accounts (id, balance) VALUES
(1, 500.00),
(2, 750.00),
(3, 1200.00);
```

3. Simulate Concurrent Transactions

Transaction T1 (Session 1)

```
BEGIN;
UPDATE accounts SET balance = balance + 100 WHERE id = 1;
-- not committed
```

Transaction T2 (Session 2)

```
SELECT SUM(balance) FROM accounts;
```

At this point, `SUM()` still uses the old value for `id = 1` because the update in Session 1 (T1) hasn't been committed.

SUM(balance) = 2450.00

4. Add COMMIT to T1 and Re-run

Transaction T1 (Session 1)

```
BEGIN;
UPDATE accounts SET balance = balance + 100 WHERE id = 1;
COMMIT;
```

Transaction T2 (Session 2)

```
SELECT SUM(balance) FROM accounts;
```

Now, `SUM()` uses the new value for `id = 1` because the update in T1 has been committed.

SUM(balance) = 2550.00

5. Conclusion:

- If T1 is not committed, T2 sees old values.
- If T1 is committed, T2 sees updated values.
- Proper transaction management ensures consistent query results.

4.2 Assignment: Simulate The Unrepeatable Read Problem

 Due Date on 14/6/2025

- Read page 752 on the primary book (FUNDAMENTALS OF Database Systems)
- See the requirement about the structures of the lab [here](#)
- You can use any SQL editor (Online or Local), but be cautious — **transaction statements may vary slightly between different DBMSs**, and the use of `DO SLEEP()` depends on the specific database engine. For example, `DO SLEEP()` is supported in MySQL 8.0.29+, while in other systems or older versions, you may need to use `SELECT SLEEP()` or an alternative approach.

4.2.1 What to assign:

1. A paragraph describing the scenario you will use to simulate the Unrepeatable Read problem.
2. A screenshot of the table you created for the simulation.
3. A screenshot showing the result of the first transaction after execution.
4. A screenshot showing the result of the second transaction after execution.
5. A paragraph discussing what happened, and how it demonstrates the Unrepeatable Read issue.